

PROJECT-BASED LEARNING

**NL2SQL-Chain: Fine-Tuned Query Generation with Version-
Controlled, Human-Gated Writes**

PHASE-I EVALUATION

Team ID: OSDBMS-V-2026-T032

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: OSDBMS-V-2026-T032
Semester: 5th
Section: A/C/E
Domain: OS and DBMS

Team Members

- | | |
|-------------------------|----------|
| 1. Vanshika Kriti Singh | 24022020 |
| 2. Anunay Sharma | 24021469 |
| 3. Sarthak Singh | 24021521 |

Mentor

Dr.Santosh Kumar Interactions completed: 3

Problem Statement & Motivation

Problem Statement

Non-technical domain experts struggle to structurally and safely export data into production-grade databases without knowing SQL. Existing NL2SQL models often prioritize raw SQL generation over operational safety and lack mandatory human review for schema-altering operations.

Motivation

- For a domain expert who is not familiar with technical database management systems and does not know SQL, can now safely and structurally export their thoughts and data into a production grade database software using just their natural language.
- Regardless of their native language, past experience in SQL, high-end powerful computers or any premium AI charges or subscription.
- Making a free, truly-local, easy to run, secure and visual solution to this problem is the motivation

Target Users / Stakeholders

- Users operating on a budget
- Business Analysts & Data Analysts
- Consumer grade hardware, not necessarily dedicated GPUs
- Small Business Owners & Entrepreneurs
- Freelance Consultants & Data Strategists
- Healthcare & HR Administrators
- C-Level Executives (CEOs, CFOs, CMOs)

Objectives

Project Objectives

- **Context-Aware SQL Generation**
- **Dialect Normalization**
- **Human in the loop for Mutation Routing**
- **Diagram supported command visualization**
- **Concurrency & Deadlock Prevention**
- **Git style VCS** (Version Control System)
- **Comprehensive logging for events and changes**

Key Objective

Build a local, cost-free, and secure Natural Language to SQL pipeline enabling non-technical users to interact with databases without requiring SQL expertise, paid cloud APIs.

Proposed Solution

Proposed Approach

1. **Schema-Linked Generation:** Uses embedding models to expose only relevant table schemas to the LLM via RAG, preventing the model from viewing the entire database.
2. **Sanitization & Dialect Normalization:** Validates generated queries against static anti-patterns and translates them into target database dialects including MySQL, Oracle, SQL Server, and Access.
3. **Dual-Path Execution Routing:** Routes safe, read-only queries directly to an isolated sandbox replica without touching live data.
4. **Human-Gated Approval:** Diverts mutating (data-altering) queries to an Entity-Relationship (ER) diagram visualizer for mandatory human verification.
5. **Checkpointed Execution:** Stores verified commands inside a secure transaction checkpoint before applying changes to the production DBMS.
6. **Cryptographic Version Control:** Tracks all database updates within a content-addressable storage tree using cryptographic hashing (Dolt), guaranteeing instant rollback and state reversibility.

Key Features

- 16-stage pipeline with EXPLAIN-style cost estimators.
- Dialect normalization compatible with MySQL, Oracle, SQL Server, and Access.
- RAG-based schema linking.
- Human-gated writes using an ER impact visualizer.
- Independent, cross-cutting audit log tracking every stage.

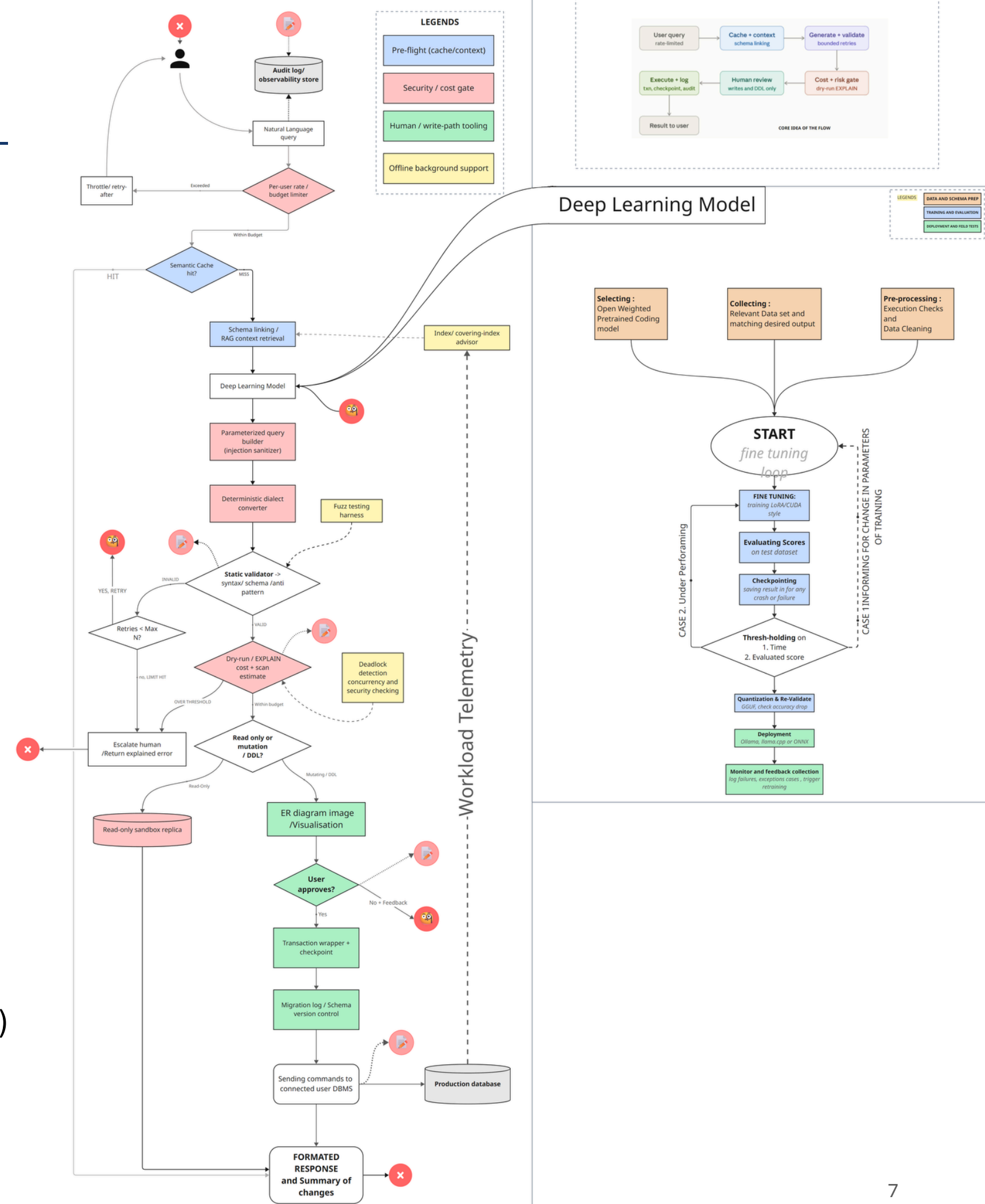
Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Operating Systems	Process management, memory management, and deadlock detection, Process & Resource Throttling,I/O Management & Write-Once File Systems, Protection, Access Matrix & Dual-Mode Operation, Mass Storage Structures & Content-Addressed File Systems, inter process communication	2. Per-User Rate Limiter 3. Audit Logger 11. Deadlock, Concurrency & Security Checking 13. Read-Only Sandbox Replic 16. DBMS Socket Communication Layer 17. Version Control System
DBMS	SQL generation, transaction control, and indexing, Query & Buffer Pool Caching, SQL Injection Prevention, SQL Dialect Translation, SQL Parsing & Semantic Analysis, Cost-Based Query Optimization (CBO)	4. Semantic Cache 5. RAG-Based Schema Linker 7. Parameterized Query Sanitizer 8. Multi-Dialect Normalizer 9. Static Validator & Anti-Pattern Detector 10. EXPLAIN-Style Cost Estimator 12. ER Diagram Impact Visualizer
Machine Learning	Natural language processing, Fine tuning LLM, Human in the loop, GUI designing	1. Natural Language Query Intake 6. DL SQL Generator 7. Human Approval Gateing 8. Software Harness

System Architecture / Workflow

Major Components

- Module 1: Natural Language Query Intake
- Module 2: Per-User Rate Limiter
- Module 3: Independent Observability Store & Audit Logger
- Module 4: Semantic Cache
- Module 5: RAG-Based Schema Linker
- Module 6: Deep Learning SQL Generator
- Module 7: Parameterized Query Sanitizer
- Module 8: Multi-Dialect Converter & Normalizer
- Module 9: Static Validator & Anti-Pattern Detector
- Module 10: EXPLAIN-Style Cost Estimator
- Module 11: Deadlock, Concurrency & Security Checker
- Module 12: Dual-Path Execution Router
- Module 13: Read-Only Sandbox Replica
- Module 14: ER Diagram Visualizer & Human Approval Gate
- Module 15: Transaction Wrapper & CAS Cryptographic Checkpoint (Dolt)
- Module 16: Connected DBMS Dispatcher & Response Formatter:



Technology Stack

Primary Programming Language	Python (3.12+)
Secondary Languages	SQL (Multi-Dialect) & TypeScript/JavaScript
Primary Target Databases	MySQL, Oracle, Microsoft SQL Server, MS Access
Database Version Control (CAS)	Dolt
Integrated Development Environment (IDE)	VS Code / DeepSeekHarness
Code Version Control	Git & GitHub

Team Contribution

Member	Role	Contribution
Vanshika Kriti Singh	Lead, QA tester, Graphic Designer, Documentatoin	Created the Docs/ README.md Created the Project Guide Divided the role and modules for each member
Sarthak Singh	Software/Harness Developer, Security implementer,	Divided the modules and a rough idea for the actual implementation of the project.
Anunay Sharma	Machine learning and AI engineer, QA tester, Socket developer	Divided the modules and a rough idea for the actual implementation of the project.

Expected Outcomes & References

Expected Outcomes

- A secure NL2SQL pipeline featuring multi-dialect SQL normalization.
- An AI generation layer optimized with semantic caching and RAG.
- A strict risk routing mechanism with an ER impact visualizer.
- An independent, cross-cutting audit log tracking every pipeline stage.
- Secure and robust system With reversible outcomes and changes

References

Repo: GITHUB

Discussion: WHITEBOARD

References

a. Research paper / journal:

1. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL
2. Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs (BIRD)
3. QLoRA: Efficient Finetuning of Quantized LLMs
4. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging
5. A Digital Signature Based on a Conventional Encryption Function (Merkle Trees) — Merkle (CRYPTO '87)
6. DeepSeek-Coder: When the Large Language Model Meets Open Group
7. NIST SP 800-53 Rev. 5: Security and Privacy Controls — National Institute of Standards and Technology

a. Dataset / repository:

8. taoyds/spider — Cross-domain complex semantic parsing and Text-to-SQL benchmark
9. AlibabaResearch/DAMO-ConvAI (BIRD) — Large-scale real-world database Text-to-SQL benchmark
10. dolthub/dolt — Git for data: SQL database with versioning, branching, and merge capabilities
11. tobymao/sqlglot — 21+ SQL dialect parser, transpiler, and anti-pattern validator
12. defog-ai/sqlcoder — Open-weights foundation models for enterprise text-to-SQL task
13. xyflow/xyflow — Interactive node-based ER diagram visualizer and workflow canvas

THANK YOU!

any questions?